26. Visitor Design Pattern in Java

26.1 Introduction

The **Visitor Pattern** is a **behavioral design pattern** that lets you define a new operation without changing the classes of the elements on which it operates.

26.2 Problem it Solves

Avoids modifying existing object structures when new behaviors are needed. Promotes the **open/closed principle**.

26.3 What is Visitor Pattern?

- Separates operations from objects
- Object structure is stable
- Visitor contains operations

26.4 Real-World Analogy

A **tax officer** visiting different types of properties (house, shop, farm) to calculate tax differently for each type.

26.5 Structure

```
package com.mannemlokesh;

public interface Visitor {
	void visit(ElementA a);

	void visit(ElementB b);
}

interface Element {
	void accept(Visitor visitor);
}

class ElementA implements Element {
	public void accept(Visitor visitor) {
		visitor.visit(this);
	}
}

class ElementB implements Element {
	public void accept(Visitor visitor) {
		visitor.visit(this);
	}
}
```

26.6 Benefits

- Add new operations without modifying objects
- Group related behavior into a visitor
- Good for performing multiple unrelated operations

26.7 Implementation Steps

1. Define Element interface with accept() method
2. Implement concrete elements
3. Define Visitor interface with visit() methods
4. Implement concrete visitors with specific logic

26.8 Examples

Example1: Document Element Renderer

```
package com.mannemlokesh.example1;

interface DocumentElement {
	void accept(DocumentVisitor visitor);
}

interface DocumentVisitor {
	void visit(Paragraph paragraph);

	void visit(Image image);
}

class Paragraph implements DocumentElement {
	String text = "This is a paragraph.";

	public void accept(DocumentVisitor visitor) {
		visitor.visit(this);
	}
}

class Image implements DocumentElement {
	String path = "image.png";

	public void accept(DocumentVisitor visitor) {
		visitor.visit(this);
	}
}

class HTMLRenderer implements DocumentVisitor {
	public void visit(Paragraph paragraph) {
		System.out.println("<p>" + paragraph.text + "</p>");
	}

	public void visit(Image image) {
		System.out.println("<img src='" + image.path + "'/>");
	}
}

public class Example1DocumentClient {
	public static void main(String[] args) {
		DocumentElement[] elements = { new Paragraph(), new Image() };
		DocumentVisitor renderer = new HTMLRenderer();

		for (DocumentElement el : elements) {
			el.accept(renderer);
		}
	}
}
```

Output:

```
<p>This is a paragraph.</p>
<img src='image.png'/>
```

Example2: Shopping Cart with Discounts

```
package com.mannemlokesh.example2;

interface ItemElement {
	int accept(ShoppingCartVisitor visitor);
}

interface ShoppingCartVisitor {
	int visit(Book book);

	int visit(Fruit fruit);
}

class Book implements ItemElement {
	int price;
	String isbn;

	public Book(int price, String isbn) {
		this.price = price;
		this.isbn = isbn;
	}

	public int accept(ShoppingCartVisitor visitor) {
		return visitor.visit(this);
	}
}

class Fruit implements ItemElement {
	int pricePerKg, weight;
	String name;

	public Fruit(int pricePerKg, int weight, String name) {
		this.pricePerKg = pricePerKg;
		this.weight = weight;
		this.name = name;
	}

	public int accept(ShoppingCartVisitor visitor) {
		return visitor.visit(this);
	}
}

class DiscountVisitor implements ShoppingCartVisitor {
	public int visit(Book book) {
		int cost = book.price;
		if (book.price > 500)
			cost -= 50;
		System.out.println("Book ISBN: " + book.isbn + ", cost: " + cost);
		return cost;
	}

	public int visit(Fruit fruit) {
		int cost = fruit.pricePerKg * fruit.weight;
```

```
            System.out.println(fruit.name + " cost: " + cost);
            return cost;
      }
}

public class Example2ShoppingClient {
      public static void main(String[] args) {
            ItemElement[] items = { new Book(600, "1234"), new Fruit(30, 2,
"Apple") };
            int total = 0;
            ShoppingCartVisitor visitor = new DiscountVisitor();

            for (ItemElement item : items) {
                  total += item.accept(visitor);
            }
            System.out.println("Total: " + total);
      }
}
```

Output:

```
Book ISBN: 1234, cost: 550
Apple cost: 60
Total: 610
```

Example3: Code Syntax Analyzer

```
package com.mannemlokesh.example3;

interface ASTNode {
      void accept(AnalyzerVisitor visitor);
}

interface AnalyzerVisitor {
      void visit(VariableNode node);

      void visit(FunctionNode node);
}

class VariableNode implements ASTNode {
      String name = "x";

      public void accept(AnalyzerVisitor visitor) {
            visitor.visit(this);
      }
}

class FunctionNode implements ASTNode {
      String functionName = "print";

      public void accept(AnalyzerVisitor visitor) {
            visitor.visit(this);
      }
}

class StaticAnalyzer implements AnalyzerVisitor {
      public void visit(VariableNode node) {
            System.out.println("Validating variable: " + node.name);
```

```
	}

	public void visit(FunctionNode node) {
		System.out.println("Validating function: " + node.functionName);
	}
}

public class Example3AnalyzerClient {
	public static void main(String[] args) {
		ASTNode[] nodes = { new VariableNode(), new FunctionNode() };
		AnalyzerVisitor analyzer = new StaticAnalyzer();

		for (ASTNode node : nodes) {
			node.accept(analyzer);
		}
	}
}
```

Output:

```
Validating variable: x
Validating function: print
```

26.9 Visitor vs Strategy vs Command

Pattern	Purpose
Visitor	Separate algorithm from object
Strategy	Select algorithm dynamically
Command	Encapsulate a request as an object

26.10 Pros and Cons

✅ **Pros**

- Adds new operations without modifying classes
- Groups related logic
- Works well with object structure traversal

❌ **Cons**

- Requires changes when new element types are added
- Can become complex with many elements and visitors

26.11 When to Use / Not to Use

Use When:

- You need to perform unrelated operations on objects
- Objects rarely change, but operations do

Avoid When:

- You frequently add new types of elements

26.12 Summary

Feature	Description
Intent	Separate logic from object structure
Use Case	Compilers, AST analysis, shopping carts
Key Benefit	Cleanly separate operations from data models

The **Visitor Pattern** is ideal when the object structure is stable but operations over them vary and need to evolve independently.